



《AI大模型Ragent项目》——知识库文档上传接口

来自： 拿个offer-开源&项目实战

马丁

2026年03月29日 12:42

前面几篇文章把文件上传的外围防线搭建完了：Spring Boot 配置限制单文件大小和请求大小，预签名 URL + 流式上传优化内存占用，Filter 层分布式信号量限流控制并发数。这些都是在请求到达业务代码之前的保护措施。这篇文章进入业务代码本身，看看 upload 接口拿到一个合法的上传请求之后，具体做了哪些事情。

upload 接口的完整流程

在看代码之前，先用一张流程图建立全局视角。upload 接口从接收请求到返回响应，中间经过了这些环节：



整个流程可以概括为一句话：把文件存到对象存储，把文档元信息存到数据库，返回文档 ID。upload 接口不做分块、不做向量化，这些耗时操作由后续的分块接口异步触发，下一篇文章会讲。

1. 接口入口

Controller 层的代码很简单：

```
@PostMapping(value = "/knowledge-base/{kb-id}/docs/upload",
    consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
public Result<KnowledgeDocumentVO> upload(
    @PathVariable("kb-id") String kbId,
    @RequestPart(value = "file", required = false) MultipartFile file,
    @ModelAttribute KnowledgeDocumentUploadRequest requestParam) {
    return Results.success(documentService.upload(kbId, requestParam, file));
}
```

接口路径 /knowledge-base/{kb-id}/docs/upload，接收三个参数：

- kbId：路径参数，指定上传到哪个知识库
- file：文件部分，注意 required = false，因为 URL 来源不需要上传文件
- requestParam：表单参数，包含来源类型、处理模式、分块策略等配置

requestParam 用 @ModelAttribute 接收，而不是 @RequestBody，因为整个请求是 multipart/form-data 格式，表单字段和文件混在一起，不是 JSON。

请求参数的完整定义：

```
@Data
public class KnowledgeDocumentUploadRequest {

    /** 来源类型: file / url */
    private String sourceType;

    /** 来源位置 (URL) */
    private String sourceLocation;

    /** 是否开启定时拉取 */
    private Boolean scheduleEnabled;

    /** 定时表达式 (cron) */
    private String scheduleCron;

    /** 处理模式: chunk / pipeline */
    private String processMode;

    /** 分块策略: fixed_size / structure_aware, 仅 processMode=chunk 时有效 */
    private String chunkStrategy;

    /** 分块参数JSON, processMode=chunk 时必传 */
    private String chunkConfig;

    /** 数据通道 ID, 仅 processMode=pipeline 时有效 */
    private String pipelineId;
}
```

2. upload 方法核心逻辑

接下来看 KnowledgeDocumentServiceImpl#upload 方法的完整实现:

```
@Override
public KnowledgeDocumentVO upload(String kbId,
                                   KnowledgeDocumentUploadRequest requestParam,
                                   MultipartFile file) {
    KnowledgeBaseDO kbDO = knowledgeBaseMapper.selectById(kbId);
    Assert.notNull(kbDO, () -> new ClientException("知识库不存在"));

    // 1. 来源类型解析: 必须显式指定, 空值或非法值直接抛异常
    SourceType sourceType = SourceType.normalize(requestParam.getSourceType());

    // 2. 校验来源参数和定时同步配置
    validateSourceAndSchedule(sourceType, requestParam);

    // 3. 上传文件到对象存储
    StoredFileDTO stored = resolveStoredFile(
        kbDO.getCollectionName(), sourceType,
        requestParam.getSourceLocation(), file);

    // 4. 解析处理模式和分块策略
    ProcessModeConfig modeConfig = resolveProcessModeConfig(requestParam);

    // 5. 构建文档记录并入库
    KnowledgeDocumentDO documentDO = KnowledgeDocumentDO.builder()
        .kbId(kbId)
        .xxxx(xxx)
        .build();
    documentMapper.insert(documentDO);

    return BeanUtil.toBean(documentDO, KnowledgeDocumentVO.class);
}
```

和很多人印象中密密麻麻的 upload 方法不同, 重构后的代码把校验、文件处理、模式解析分别提取到了独立方法中, 主干流程变得很清晰: 解析来源 → 校验参数 → 上传文件 → 解析模式 → 建记录 → 返回。
下面逐个拆解核心步骤。

2.1 来源类型解析

SourceType 枚举定义了两种来源类型:

```
public enum SourceType {
    FILE("file"),    // 本地文件上传
    URL("url");      // 远程 URL 抓取
}
```

来源类型的解析只有一行:

```
SourceType sourceType = SourceType.normalize(requestParam.getSourceType());
```

SourceType.normalize() 做了两层校验: 空值直接抛异常 (来源类型不能为空), 非法值也抛异常 (不支持的来源类型)。也就是说 sourceType 是前端必传的参数, 接口不做自动推断。

```
public static SourceType normalize(String value) {
    if (StrUtil.isBlank(value)) {
        throw new IllegalArgumentException("来源类型不能为空");
    }
    SourceType result = fromValue(value);
    if (result == null) {
        throw new IllegalArgumentException("不支持的来源类型: " + value);
    }
    return result;
}
```

两种来源类型的使用场景不同：

来源类型	使用场景	参数要求
FILE	用户从本地选择文件上传	必须有 file 参数
URL	填写远程文件地址，由服务端抓取	必须有 sourceLocation 参数

2.2 参数校验：validateSourceAndSchedule

校验逻辑被提取到了一个独立方法中：

```
private void validateSourceAndSchedule(SourceType sourceType,
                                      KnowledgeDocumentUploadRequest request) {
    String sourceLocation = StrUtil.trimToNull(request.getSourceLocation());
    if (SourceType.URL == sourceType && !StringUtils.hasText(sourceLocation)) {
        throw new ClientException("来源地址不能为空");
    }
    if (!isScheduleEnabled(sourceType, request)) {
        return;
    }
    String scheduleCron = StrUtil.trimToNull(request.getScheduleCron());
    if (!StringUtils.hasText(scheduleCron)) {
        throw new ClientException("定时表达式不能为空");
    }
    try {
        if (CronScheduleHelper.isIntervalLessThan(scheduleCron,
            new Date(), scheduleProperties.getMinIntervalSeconds())) {
            throw new ClientException(
                "定时周期不能小于 " + scheduleProperties.getMinIntervalSeconds() + " 秒");
        }
    } catch (IllegalArgumentException e) {
        throw new ClientException("定时表达式不合法");
    }
}
```

这个方法做了两件事：

- 1. 来源参数校验：URL 类型必须有 sourceLocation
- 2. 定时同步参数校验：如果开启了定时同步，校验 cron 表达式的合法性和最小周期

定时同步是否开启，由 isScheduleEnabled 方法判断：

```
private boolean isScheduleEnabled(SourceType sourceType,
                                  KnowledgeDocumentUploadRequest request) {
    return SourceType.URL == sourceType
        && Boolean.TRUE.equals(request.getScheduleEnabled());
}
```

只有 URL 来源且 scheduleEnabled=true 时才开启。FILE 来源的文件在本地，没有远程源可拉取，定时同步没有意义。

2.3 定时同步（URL 来源）

实际企业中，很多团队把文档写在钉钉、飞书、Notion 等在线文档平台上。这些文档会不断更新，但文档作者改完内容后，通常不会主动去知识库重新上传最新版本。时间一长，知识库里的内容就慢慢过时了。如果每次文档更新都依赖人工手动重新上传，实际运行中基本不可能坚持。

所以 URL 来源支持定时拉取机制。用户上传 URL 类型的文档时，可以设置 `scheduleEnabled=true` 和一个 `scheduleCron` 表达式，系统会按照 `cron` 定义的周期自动重新抓取远程文件并触发分块更新，保持知识库数据的时效性。

`upload` 接口中对定时参数做了两层校验：

1. **表达式合法性**：`cron` 表达式格式必须正确，不合法直接拒绝
2. **最小周期限制**：防止用户设置过短的周期（比如每秒拉取一次），通过 `scheduleProperties.getMinIntervalSeconds()` 配置最小间隔

定时任务调度的底层设计（定时任务怎么注册、怎么触发、怎么和分块流程衔接）不在本篇讨论，后续文章会讲。

2.4 处理模式：resolveProcessModeConfig

处理模式的解析逻辑也被提取到了独立方法，返回一个 `ProcessModeConfig` record：

```
private record ProcessModeConfig(ProcessMode processMode, ChunkingMode chunkingMode,
                                String chunkConfig, String pipelineId) {

}

private ProcessModeConfig resolveProcessModeConfig(
    KnowledgeDocumentUploadRequest request) {
    ProcessMode processMode = ProcessMode.normalize(request.getProcessMode());
    if (ProcessMode.CHUNK == processMode) {
        ChunkingMode chunkingMode = ChunkingMode.fromValue(request.getChunkStrategy());
        String chunkConfig = buildChunkConfigJson(chunkingMode, request);
        return new ProcessModeConfig(processMode, chunkingMode, chunkConfig, null);
    } else {
        if (!StringUtils.hasText(request.getPipelineId())) {
            throw new ClientException("使用Pipeline模式时，必须指定Pipeline ID");
        }
        try {
            ingestionPipelineService.get(request.getPipelineId());
        } catch (Exception e) {
            throw new ClientException("指定的Pipeline不存在: " + request.getPipelineId());
        }
        return new ProcessModeConfig(processMode, null, null, request.getPipelineId());
    }
}
```

`ProcessMode` 枚举定义了两种处理模式：

```
public enum ProcessMode {
    CHUNK("chunk"),          // 分块策略模式（默认）
    PIPELINE("pipeline");    // Pipeline 管道模式
}
```

`ProcessMode.normalize()` 方法在参数为空时默认返回 `CHUNK`，也就是说如果前端不传 `processMode`，默认走分块模式。

两种模式在 `upload` 接口中的参数要求不同：

CHUNK 模式需要指定分块策略和分块配置：

`chunkStrategy`：必传，分块策略，比如 `fixed_size`（固定大小分块）或 `structure_aware`（结构感知分块），非法值直接抛异常
`chunkConfig`：分块参数的 JSON 字符串，比如 `{"targetChars":1400,"maxChars":1800,"minChars":600,"overlapChars":0}`

PIPELINE 模式需要指定 Pipeline ID：

`pipelineId`：必填，指定使用哪个预定义的 Pipeline。方法内会校验这个 Pipeline 是否存在

用 `record` 封装返回值的好处是，主方法里不需要定义一堆零散的局部变量（`chunkingMode`、`chunkConfig`、`pipelineId`），直接通过 `modeConfig.xxx()` 访问，结构更清晰。

2.5 PIPELINE 模式简介

PIPELINE 模式是一种节点编排的处理方式，类似 ETL 的思路。它把文档处理拆成多个可组合的节点——解析节点、清洗节点、分块节点、Embedding 节点等——通过 Pipeline 定义把这些节点串联起来执行。

和 CHUNK 模式的区别在于：CHUNK 模式是代码中固定的流程（提取文本 → 分块 → Embedding），流程是写死的，用户只能调整分块策略和参数。而 PIPELINE 模式允许用户自定义节点组合和执行顺序，比如在分块之前加一个文本清洗节点，或者替换默认的解析器，灵活度更高。

在 upload 接口中，PIPELINE 模式只需要指定一个 pipelineId，接口会校验这个 Pipeline 是否存在，然后把 ID 记录到文档记录中。具体的 Pipeline 架构、节点定义和执行引擎，后续文章会详细讲。

2.6 构建文档记录并入库

最后一步是构建 KnowledgeDocumentDO 对象并插入数据库。核心字段的取值逻辑：

- docName: 取自对象存储返回的原始文件名 stored.getOriginalFilename()
- fileUrl: 文件在对象存储中的地址
- fileType: 文件类型标识，比如 pdf、markdown、docx
- fileSize: 文件大小（字节）
- status: 固定设为 PENDING，表示文件已上传但尚未分块
- sourceLocation: 只有 URL 来源才记录，FILE 来源设为 null
- scheduleEnabled / scheduleCron: 通过 isScheduleEnabled() 判断，只有 URL 来源且开启定时同步时才写入
- chunkCount: 初始为 0，分块完成后更新

插入数据库后，通过 BeanUtil.toBean 把 DO 转成 VO 返回给前端。其实不返回也行，只不过最早没写前端时，返回数据我能拿着文档 ID 执行下面的分块，不用再去数据库复制了，哈哈。

文件上传到对象存储

文件上传是 upload 接口中最有技术含量的部分。不同的来源类型，处理方式差异很大。

1. resolveStoredFile 分发逻辑

resolveStoredFile 方法根据来源类型分发到不同的处理逻辑：

```
private StoredFileDTO resolveStoredFile(String bucketName, SourceType sourceType,
                                       String sourceLocation, MultipartFile file) {
    if (SourceType.FILE == sourceType) {
        Assert.notNull(file, () -> new ClientException("上传文件不能为空"));
        return fileStorageService.upload(bucketName, file);
    }
    return remoteFileFetcher.fetchAndStore(bucketName, sourceLocation);
}
```

逻辑很清晰：FILE 类型走 fileStorageService.upload 直传，URL 类型走 remoteFileFetcher.fetchAndStore 远程抓取。

注意这里的一个设计：远程文件抓取的逻辑没有直接写在 KnowledgeDocumentServiceImpl 里，而是提取到了独立的 RemoteFileFetcher 服务中。这样做的好处是，定时同步场景也需要远程抓取文件，提取出来后两个场景可以复用。

上传成功后返回 StoredFileDTO，包含四个字段：

字段	类型	说明
url	String	文件在对象存储中的访问地址
detectedType	String	检测到的 MIME 类型（如 application/pdf）
size	Long	文件大小（字节）
originalFilename	String	原始文件名

2. FILE 类型：本地文件上传

FILE 类型的处理最简单，一行代码搞定：

```
return fileStorageService.upload(bucketName, file);
```

文件在到达这里之前，已经经过了多层保护：

- Filter 层的分布式信号量限流控制了并发数
- Spring Boot 的 `max-file-size` 配置限制了单文件大小
- 预签名 URL + 流式上传避免了大文件占满内存

这里只需要把 `MultipartFile` 上传到对象存储即可。`fileStorageService.upload` 内部会通过预签名 URL 流式上传到 S3 兼容存储（RustFS），不会把整个文件读到内存中。这部分逻辑在前面的文章里已经讲过。

3. URL 类型：远程文件抓取

URL 类型的处理复杂得多。用户不上传文件，而是填一个远程地址，由服务端去抓取文件内容并上传到对象存储。

这里有一个关键规则：`sourceLocation` **必须是文件的直接下载链接，不能是网页地址**。服务端拿到这个 URL 后会直接发 GET 请求下载文件内容，如果 URL 指向的是一个 HTML 页面而不是文件本身，下载到的就是 HTML 源码，后续解析和分块都会出问题。

举个例子，GitHub 上的文件有两种 URL：

网页地址：`https://github.com/bytedance/deer-flow/blob/main/docs/SKILL_NAME_CONFLICT_FIX.md`——这是 GitHub 的文件浏览页面，打开是一个带导航栏、侧边栏的 HTML 页面，不是文件本身

原始文件地址：`https://raw.githubusercontent.com/bytedance/deer-flow/main/docs/SKILL_NAME_CONFLICT_FIX.md`——这才是 Markdown 文件的原始内容，GET 请求返回的就是纯文本

用户需要填的是第二种。类似的，如果要抓取一个 PDF，URL 应该是点击后直接开始下载的那个链接，而不是一个包含预览界面的页面。

简单的判断方法：在浏览器中直接访问这个 URL，如果看到的是文件内容（或者浏览器弹出下载框），就是对的；如果看到的是一个带有页面布局的网页，就需要找到真正的文件下载链接。

这部分逻辑封装在 `RemoteFileFetcher` 服务中，来看核心的 `fetchAndStore` 方法：

```
@Slf4j
@Component
@RequiredArgsConstructor
public class RemoteFileFetcher {

    private final HttpClientHelper httpClientHelper;
    private final FileStorageService fileStorageService;

    @Value("${spring.servlet.multipart.max-file-size:50MB}")
    private DataSize maxFileSize;

    public StoredFileDTO fetchAndStore(String bucketName, String url) {
        long maxBytes = maxFileSize.toBytes();
        url = url.trim();

        // 1. HEAD 探测
        HttpClientHelper.HttpHeadResponse headResponse = tryHead(url);
        Long headContentLength = headResponse == null ? null : headResponse.contentLength();
        checkSizeLimit(maxBytes, headContentLength);

        // 2. 流式下载 → 临时文件 → 上传到对象存储
        try (HttpClientHelper.HttpFetchStream response =
            httpClientHelper.openStream(url, Map.of(), maxBytes)) {
            String fileName = firstHasText(
                response.fileName(),
                headResponse == null ? null : headResponse.fileName(),
                "remote-file");
            String contentType = firstHasText(
                response.contentType(),
                headResponse == null ? null : headResponse.contentType(),
                null);
            return uploadViaTemp(
                bucketName, response.bodyStream(),
```

```
        fileName, contentType, maxBytes);  
    }  
}  
}
```

流程分两步，逐步拆解。

3.1 HEAD 探测

第一步是发一个 HTTP HEAD 请求到远程地址。HEAD 请求只获取响应头，不下载文件内容，开销很小。通过响应头可以拿到：

Content-Length: 文件大小
Content-Type: 文件类型 (MIME)
Content-Disposition: 可能包含文件名

拿到 Content-Length 后，先做一次文件大小校验。如果远程文件超过了 max-file-size 的限制，直接拒绝，不浪费带宽去下载。但 HEAD 请求不一定能成功。有些服务器不支持 HEAD 方法，有些 CDN 会拦截 HEAD 请求。所以 tryHead 方法用 try-catch 包裹，HEAD 失败了就返回 null，跳过这一步，直接进入流式下载：

```
private HttpClientHelper.HttpHeadResponse tryHead(String url) {  
    try {  
        return httpClientHelper.head(url, Map.of());  
    } catch (Exception e) {  
        log.debug("HEAD 获取失败, 改为直接下载: {}", url, e);  
        return null;  
    }  
}
```

HEAD 探测是一个优化手段，不是必须的。它的价值在于：如果远程文件明显超限（比如一个 500MB 的文件），HEAD 请求几毫秒就能拦住，不需要开始下载后再中断。

3.2 流式下载并通过临时文件上传

第二步是打开流式连接，从远程服务器下载文件内容，先写到本地临时文件，再从临时文件上传到对象存储。

你可能会想：如果 HEAD 响应已经返回了 Content-Length，为什么不直接把远程流传给 S3，而要绕一圈走临时文件？

这里有一个实际踩过的坑：**部分源站的 Content-Length 响应并不可靠**。有些服务器返回的 Content-Length 和实际传输的字节数不一致（比如动态压缩、传输中断等情况），而 S3 的 PutObject API 要求上传的字节数必须和声明的 Content-Length 完全一致，不一致就会报错。

所以 RemoteFileFetcher 做了一个设计决策：**远程文件统一先落临时文件，以实际读取到的字节数作为上传大小**。这样不管源站返回的 Content-Length 是否准确，都不会出问题。

```
private StoredFileDTO uploadViaTemp(String bucketName, InputStream remoteStream,  
                                     String fileName, String contentType,  
                                     long maxBytes) {  
  
    Path tempFile = null;  
    try {  
        tempFile = Files.createTempFile("knowledge-upload-", ".tmp");  
        long size = copyWithLimit(remoteStream, tempFile, maxBytes);  
        if (size == 0) {  
            throw new ClientException("远程文件内容为空");  
        }  
        try (InputStream tempInputStream = Files.newInputStream(tempFile)) {  
            return fileStorageService.upload(  
                bucketName, tempInputStream, size, fileName, contentType);  
        }  
    } catch (IOException e) {  
        throw new ServiceException("远程文件上传失败: " + e.getMessage());  
    } finally {  
        if (tempFile != null) {  
            try {  
                Files.deleteIfExists(tempFile);  
            } catch (IOException e) {  
                log.warn("删除远程上传临时文件失败: {}", tempFile, e);  
            }  
        }  
    }  
}
```



```
        }
    }
}
}
```

流程是：

- 1. 创建临时文件（Files.createTempFile）
- 2. 把远程流的数据写到临时文件中，同时做大小限制检查——如果超过 maxBytes，中断写入并抛异常
- 3. 从临时文件打开一个新的输入流，连同已知的实际大小一起上传到对象存储
- 4. 无论成功还是失败，finally 块中删除临时文件

这个方案的代价是多了一次磁盘 IO（先写临时文件再读出上传），但换来的是可靠性：不依赖源站返回的 Content-Length，以实际下载到的字节数为准。对于远程文件抓取这种本身就耗时的操作，多一次本地磁盘 IO 的开销可以忽略不计。

3.3 文件名解析

远程文件的文件名需要从响应头中解析。RemoteFileFetcher 用一个 firstHasText 工具方法按优先级取值：

```
String fileName = firstHasText(
    response.fileName(),           // GET 响应的文件名
    headResponse == null ? null : headResponse.fileName(), // HEAD 响应的文件名
    "remote-file");               // 兜底默认值
```

文件名的来源有两个：Content-Disposition 响应头（如 attachment; filename="report.pdf"），以及 URL 路径中的最后一段（如 https://example.com/docs/report.pdf 中的 report.pdf）。HttpClientHelper 内部的 resolveFileName 方法会按这个优先级解析。如果两个都拿不到，就用 remote-file 作为兜底文件名。虽然文件名不影响后续的分块处理，但在文档列表页面展示时，一个有意义的文件名比 remote-file 的用户体验要好得多。

3.4 为什么把远程抓取提取到 RemoteFileFetcher

把远程文件抓取的逻辑从 KnowledgeDocumentServiceImpl 提取到独立的 RemoteFileFetcher 服务，不只是为了让 upload 方法更简洁。更重要的原因是**复用**。

项目中有两个场景需要远程文件抓取：

- 1. **用户上传**：用户填写 URL，upload 接口调用 fetchAndStore() 抓取文件并上传到对象存储
- 2. **定时同步**：定时任务周期性地重新抓取远程文件，此时需要检测文件是否有变化

RemoteFileFetcher 除了 fetchAndStore() 方法外，还提供了 fetchIfChanged() 方法，专门用于定时同步场景。它通过 ETag、Last-Modified 和 SHA-256 内容哈希三种机制检测文件是否有变化，如果文件没变就跳过，避免不必要的重复处理。这两个方法共享了 HEAD 探测、大小限制、临时文件处理等底层逻辑，提取到一个类中是自然的选择。

FAQ

1. upload 方法为什么没加事务

看完代码你可能注意到了，upload 方法上没有 @Transactional 注解。方法里既有对象存储操作（上传文件），又有数据库操作（插入文档记录），为什么不用事务包起来？
原因很直接：**事务内不建议做外部 IO 操作（尤其是长耗时）**。
文件上传到对象存储是一个网络 IO 操作，耗时取决于文件大小和网络状况，可能是几秒，也可能是几十秒。如果把这个操作放在数据库事务中，事务会一直持有数据库连接不释放，直到文件上传完成。
假设 upload 接口的并发量是 20，每个请求上传耗时 10 秒，那就是 20 个数据库连接被占用 10 秒。如果连接池大小是 20，其他所有需要数据库的请求都会被阻塞。这就是典型的长事务导致连接池耗尽的问题。
所以 upload 方法故意不加事务，让对象存储操作和数据库操作各自独立执行。

2. 文件上传成功但数据库失败怎么办

不加事务带来的代价是：如果文件已经上传到对象存储，但随后的数据库插入失败了（比如数据库宕机、唯一索引冲突），对象存储中就会留下一个没有数据库记录引用的孤儿文件。
这种情况在实际生产中发生的概率很低——数据库插入一条记录很少会失败。但防御性设计还是有必要的。
应对方案是定时任务清理：

```
定时扫描对象存储中的文件列表
和数据库的 file_url 字段做比对
```

找出在对象存储中存在、但数据库中没有记录引用的文件

加一个时间窗口保护（比如只清理创建时间超过 1 小时的孤儿文件），避免误删正在上传中的文件

定期执行清理

这个方案的思路是最终一致性：允许短时间内存在不一致（孤儿文件），但通过定时任务保证最终一致。对于内部知识库这种场景，偶尔多占一点对象存储空间完全可以接受。

3. 为什么不直接存 MIME 类型

你可能注意到，fileType 字段存的是 pdf、markdown、docx 这样的简短标识，而不是 application/pdf、text/markdown 这样的标准 MIME 类型。为什么要自己做一层转换？

原因有几个：

1. **同一种文件格式对应多个 MIME 类型。**比如 PDF 文件，不同浏览器或服务器可能返回 application/pdf、application/x-pdf，Markdown 可能是 text/markdown 或 text/x-markdown，Word 文档的 MIME 更是五花八门。如果直接存 MIME，后续按文件类型查询或过滤时，就得写一堆 IN ('application/pdf', 'application/x-pdf') 这样的条件，维护成本很高。
2. **可读性和展示友好。**前端页面展示文件类型时，用户看到 pdf 比看到 application/pdf 直观得多。数据库里查数据排查问题时也一样。
3. **后续分块策略可能依赖文件类型做路由。**比如 PDF 走 Tika 解析，Markdown 走专门的 Markdown 解析器。用统一的简短标识做路由条件，比用 MIME 类型简洁。

FileTypeDetector 的转换逻辑也很简单：优先用文件扩展名匹配，匹配不到再用 MIME 类型匹配，都匹配不到就用扩展名本身兜底。

4. 远程文件抓取的鉴权限制

当前的 RemoteFileFetcher 只支持两种场景：完全公开的 URL（不需要鉴权），以及通过 URL 参数携带 Token 的简单鉴权（比如 https://example.com/doc.pdf?token=xxx）。这两种情况下，直接发 GET 请求就能拿到文件内容。

但实际企业中，很多文档存放在需要复杂鉴权的平台上，比如钉钉文档、飞书文档、Confluence 等。这些平台的鉴权流程各不相同——有的需要 OAuth 授权拿 Access Token，有的需要先调 API 获取文档导出链接，有的还有 Cookie 鉴权和 CSRF 校验。想要支持这些平台，就需要针对每个平台做专门的适配：实现各自的授权流程、调用平台特定的 API 获取文档内容。

这属于常规的平台适配工作，不涉及 RAG 核心链路的设计，不在本系列的讨论范围内。如果你的知识库需要对接这些平台，可以在 RemoteFileFetcher 的基础上扩展，按平台类型分发到不同的抓取实现。

小结

回过头看，upload 接口的逻辑并不复杂，但里面有不少值得注意的设计取舍：

upload 只管存文件和建记录，状态设为 PENDING，不触发分块——上传和分块的职责拆开，给用户更多控制空间

FILE 直传走 fileStorageService，URL 远程抓取走 RemoteFileFetcher，统一先落临时文件再上传，不依赖源站返回的 Content-Length

URL 来源支持 cron 定时拉取，解决在线文档更新后知识库内容过时的问题

CHUNK 和 PIPELINE 两种处理模式在上传时只记录参数，实际分块是后续单独触发的

不加事务是故意的，事务内做网络 IO 会拖长连接占用时间，孤儿文件靠定时任务兜底清理

文件存好了，记录也建好了，接下来就是把文档拆成 chunk、生成向量、写入向量库。下一篇讲分块触发和执行的完整链路。